

ADVANCED MULTI AI AGENT 2026



스스로 만들어지는 **개인 지식 베이스**

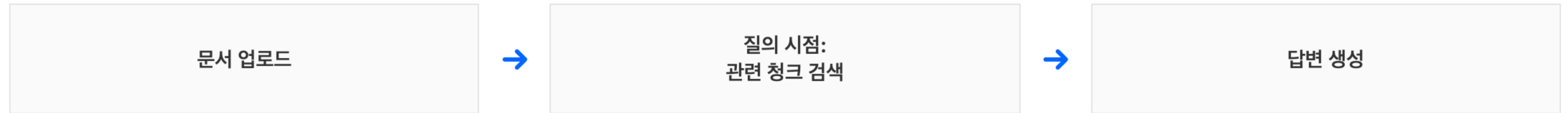
LLM Wiki — RAG를 넘어, LLM이 위키를 짓고 계속 최신으로 유지한다

Karpathy 패턴 | nashsu 구현 | GPL-3.0 | Tauri v2

박경규

문제 — RAG는 매번 처음부터

대부분의 'LLM + 문서' 경험은 RAG다



매 질문마다 이 과정을 처음부터 반복

01 축적 없음

지식이 쌓이지 않음

매 질문마다 지식을 처음부터 다시 발견

02 종합의 벽

조각을 매번 재조립

5개 문서를 종합해야 하는 질문마다 조각을 다시 이어 붙임

03 대표 사례

같은 방식의 도구들

NotebookLM · ChatGPT 파일 업로드도 같은 방식

LLM이 매번 지식을 처음부터 다시 발견한다 — 아무것도 쌓이지 않는다

핵심 아이디어 — 복리로 쌓이는 위키

질의 시점 검색 대신, 지속적으로 갱신되는 위키를 구축한다

위키는 지속적으로 복리로 쌓이는 산출물(compounding artifact)이다.

인제스트 때 LLM이 하는 일

- 소스를 읽고 핵심 추출
- 엔티티·요약 페이지 갱신
- 새 데이터가 기존 주장과 모순되면 표시
- 종합(synthesis)을 강화·수정

역할 구도

Obsidian = IDE · LLM = 프로그래머 · 위키 = 코드베이스

- 사람 = 큐레이션·질문 / LLM = 요약·상호참조·정리·기록

소스를 더할수록, 질문할수록 위키는 더 풍부해진다

RAG vs LLM Wiki

무엇이 근본적으로 다른가

항목	RAG	LLM Wiki
지식 처리	질의마다 재발견	한 번 컴파일 후 유지
산출물	없음 (휘발)	영속 위키 (복리 축적)
상호 참조	매번 재구성	이미 존재
모순 처리	발견되지 않음	사전에 표시됨
저장소	벡터 인덱스	마크다운 git 저장소
탐색 방식	임베딩 검색	index.md + 지식 그래프

질문의 좋은 답도 위키에 페이지로 저장 → 탐색까지 복리로 쌓인다

아키텍처 개요

사람이 큐레이션하고 · LLM이 위키를 짓고 계속 유지한다



SCHEMA **설정(Schema)**이 이 파이프라인 전체를 규율한다 — **3계층 구조·역할·규칙**은 다음 장에서

좋은 질의 답변도 위키에 페이지로 저장 → **지식이 복리로 축적된다**

3계층 아키텍처

Raw Sources → Wiki → Schema

RAW SOURCES

원본 (불변)

- 기사·논문·이미지·데이터
- LLM은 읽기만, 수정 안 함
- 진실의 원천



WIKI

LLM 생성·유지

- entities/ · concepts/ · sources/ · synthesis/ · comparisons/
- index.md · log.md · overview.md
- 사람은 읽고, LLM이 쓴다



SCHEMA

규칙·설정

- CLAUDE.md
- AGENTS.md
- 구조·규칙·워크플로우 정의
- "일반 챗봇 → 규율 있는 위키 관리자"

핵심 설정은 **Schema** — 사람과 LLM이 함께 진화시킨다

3대 오퍼레이션

Ingest · Query · Lint

INGEST

수집

- 소스 읽기 → 요약 페이지 작성 → index 갱신 → 엔티티·개념 갱신 → log 기록

한 소스가 10~15개 페이지를 건드림

QUERY

질의

- 관련 페이지 검색·읽기·종합(인용 포함)
- 좋은 답은 다시 위키에 페이지로 저장 → 탐색도 복리로 축적

LINT

점검

- 모순·낱은 주장·고립 페이지·누락된 상호참조·데이터 공백을 주기적으로 점검

수집·질의·점검의 순환이 위키를 살아 있게 한다

index.md & log.md

위키를 항해하는 두 개의 특수 파일

콘텐츠 지향

index.md

- 모든 페이지의 카탈로그
- 각 페이지: 링크 + 한 줄 요약 + 메타데이터
- 카테고리별 정리(entities·concepts·sources)
- 질의 시 index를 먼저 읽고 관련 페이지로 파고들

시간 지향

log.md

- append-only 작업 기록 (인제스트·질의·린트)
- 일관된 접두어를 쓰면 grep으로 파싱 가능

```
## [2026-04-02] ingest | 기사 제목  
$ grep "^## \[" log.md | tail -5
```

소스 **~100개** · 페이지 **수백 개** 규모까지 임베딩 RAG 인프라 없이도 충분

적용 사례

시간에 걸쳐 지식을 쌓는 모든 곳



개인 성장

목표·건강·심리, 저널·기사·팟캐스트 정리



장기 리서치

수주~수개월, 논문·리포트로 진화하는 논지 구축



책 읽기

챕터별 정리, 인물·테마·플롯 위키 (팬 위키처럼)



팀 내부 위키

Slack·회의록·문서로 LLM이 유지



경쟁 분석·실사

흩어진 자료를 구조화



강의 노트·취미 딥다이브

지식이 흩어지지 않고 정리됨

사람은 **소스 큐레이션과 좋은 질문**, LLM은 나머지 전부

왜 되는가 — 유지 비용이 0에 가까워서

지식 베이스가 죽는 진짜 이유

지식 베이스의 고통은 읽기·사고가 아니라 부기(bookkeeping)다.

사람의 한계

상호 참조 갱신, 요약 최신화, 모순 표시, 수십 페이지의 일관성 유지 — 사람은 **유지 부담이 가치보다 빨리 커져** 위키를 방치한다.

LLM이 바꾸는 것

- LLM은 지치지 않는다
- 상호 참조를 잊지 않는다
- 한 번에 15개 파일을 손댄다
- → **유지 비용이 0에 가깝다**

MEMEX (1945)

Vannevar Bush가 그린 개인 지식 저장소 — 문서 사이의 **연관 경로가 문서만큼 중요**하다고 봤다.

그가 끝내 못 푼 문제는 "누가 유지하는가"였다. 그 답을 이제 **LLM이 말한다.**

개념에서 앱으로 — nashsu/llm_wiki

Karpathy의 추상 패턴을 크로스 플랫폼 데스크톱 앱으로 구현·확장

원본에서 유지한 것

- 3계층 · 3대 작업
- index.md · log.md
- [[wikilink]] 상호참조
- YAML frontmatter
- Obsidian 호환
- 사람 큐레이션 + LLM 유지

확장한 것

- 3열 데스크톱 UI (Tree · Chat · Preview)
- purpose.md — "[위키의 영혼](#)"(왜 존재하는가)
- 2단계 CoT 인제스트
- 지식 그래프 + Louvain
- Deep Research · Web Clipper · MCP

기술 스택 **Tauri v2 (Rust)** · React 19 + TypeScript · GPL-3.0 · 다중 provider(OpenAI · Anthropic · Google · Ollama)

2단계 Chain-of-Thought 인제스트

읽기와 쓰기를 두 번의 LLM 호출로 분리 → 품질 향상

STEP 1 · 분석

소스를 읽고 구조화

- 소스를 읽고 구조화 분석
- 핵심 엔티티 · 개념 · 주장
- 기존 위키와의 연결
- 모순 · 긴장
- 위키 구조 권장



STEP 2 · 생성

페이지를 쓰고 갱신

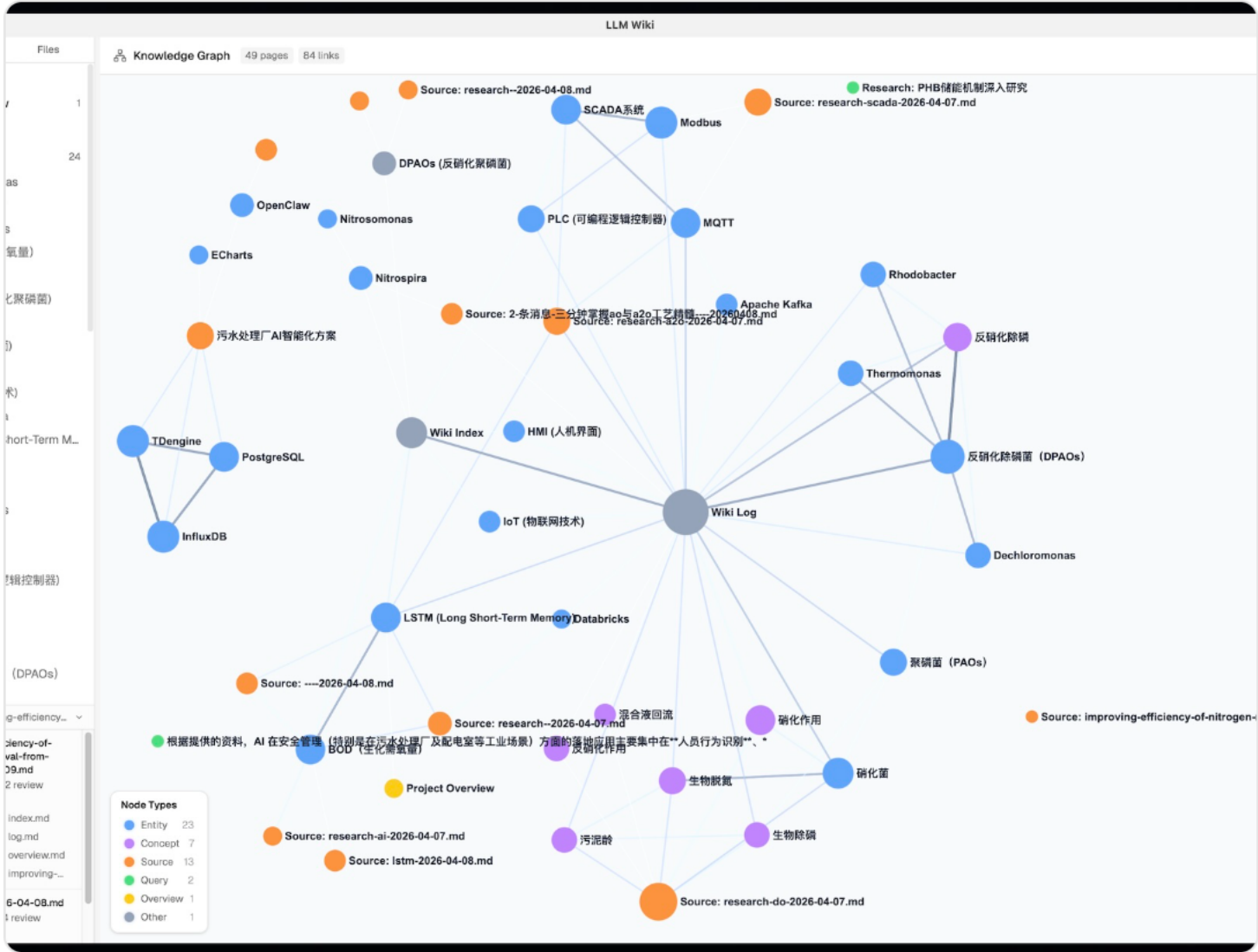
- frontmatter 포함 소스 요약
- 상호참조 엔티티 · 개념 페이지
- index · log · overview 갱신
- 리뷰 항목
- Deep Research 검색 쿼리

추가 개선 SHA256 증분 캐시(미변경 파일 스킵) · 영속 인제스트 큐(직렬 · 재시도 3회) · 출처 추적성 sources:[]

지식 그래프

4-신호 관련성 · Louvain 커뮤니티 · 그래프 인사이트

실제 지식 그래프



LLM Wiki 앱 · 49 pages · 84 links · sigma.js + graphology + ForceAtlas2

LLM Wiki · 개념과 구현

4-신호 관련성 모델

신호	가중치
직접 링크	×3.0
출처 중복	×4.0
Adamic-Adar	×1.5
타입 친화도	×1.0

Louvain 커뮤니티

- 링크 토폴로지 기반 지식 클러스터 자동 발견
- 응집도 점수(< 0.15 경고)
- 타입 · 커뮤니티 색상 토글

그래프 인사이트

- 뜻밖의 연결(교차 커뮤니티 · 타입 엣지)
- 지식 공백(고립 · 희소 · 브리지 노드)
- 한 클릭 Deep Research

검색 파이프라인 (Query)

토큰화 → 벡터(선택) → 그래프 확장 → 예산 제어

1

토큰화 검색

영어 단어분리 / 중국어 CJK bigram, 제목 일치 **보너스(+10)**, wiki/ 와 raw/sources/ 모두 검색

2

벡터 의미 검색 (선택)

OpenAI 호환 임베딩 + LanceDB, 키워드가 안 겹쳐도 의미로 매칭. 전체 recall **58.2% → 71.4%**

3

그래프 확장

상위 결과를 seed로 **4-신호 관련성** 2-hop 탐색(깊이 감쇠)

4

예산 제어

context window 4K~1M, 비례 할당 **Wiki 60 / chat 20 / index 5 / system 15**

벡터 검색은 선택 — 끄면 **토큰화 검색 + 그래프 확장**으로 자동 fallback

생태계 — 위키를 넘어 도구·에이전트로

수집·검색을 앱 밖으로 확장

01 RESEARCH

Deep Research

Tavily · SerpApi · SearXNG 다중 쿼리 웹 검색 → 결과를 위키에 자동 인제스트

02 CLIPPER

Chrome Web Clipper

Readability.js + Turndown.js로 웹 → 마크다운, 자동 인제스트

03 API

로컬 API + MCP Server

127.0.0.1:19828, 하이브리드 검색·파일 읽기·그래프 탐색

04 AGENT

Agent Skill

Claude Code / Codex에 `npx skills add ...` 한 줄 설치

05 VAULT

Obsidian 호환

위키 디렉토리를 vault로 사용, 그래프 뷰

06 LLM

다중 provider LLM

OpenAI · Anthropic · Google · Ollama · Custom

`npx skills add ...` 한 줄로 내 지식 베이스를 에이전트에 연결

시작하기

설치 → 프로젝트 → 소스 → 질문

설치 & 빠른 시작

- 바이너리 — macOS(.dmg)·Windows(.msi)·Linux(.deb/.ApplImage)
- 소스 빌드 — Node 20+·Rust 1.70+, npm run tauri dev
- ① 프로젝트 생성(템플릿) ② Settings에서 LLM provider ③ Sources 가져오기 ④ Chat·Graph·Lint

프로젝트 구조

```
my-wiki/  
├─ purpose.md    · schema.md  
├─ raw/sources/  (불변)  
└─ wiki/  
    ├─ index.md · log.md · overview.md  
    ├─ entities/ · concepts/ · sources/  
    └─ queries/ · synthesis/ · comparisons/
```


CLOSING

지식은 한 번 컴파일하고, **계속 최신으로** 둔다.

▪ RAG → 지속 갱신 위키

▪ 사람은 질문, LLM은 유지

▪ index + 그래프면 임베딩 없이도 충분

참고 자료 · REFERENCES

Karpathy 패턴 문서 · <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>

LLM Wiki (GitHub) · https://github.com/nashsu/llm_wiki

소개 글 (PyTorch KR) · <https://discuss.pytorch.kr/t/llm-wiki-feat-karpathy-llm-wiki/10139>

THANK YOU

박경규